Creative Assignment 2: Feature Engineering

My CA1 detector of off-task behaviour used 14 clip-level averages and three contrast features built from them. For this assignment I went back to the action log behind those averages and engineered 24 new features that an average cannot express: how many actions a clip contains, the longest pause rather than the mean pause, how many widgets and skills the student touched, Bayesian Knowledge Tracing (BKT) estimates, and how slow the recent actions around each action were. The detector was evaluated with nested cross-validation grouped by student: in every split, the training students alone chose the feature set, the model and the decision threshold, and the held-out students only scored the result. For students the model has not seen, the detector with the new features reaches **a pooled Kappa of 0.381 and an AUC of 0.884**. The detector with the old features, built by the same rule, reaches 0.306 and 0.750, and my CA1 hand-in reported 0.328 and 0.777.

1 Data and Feature Engineering

1.1 Linking the two files and checking the new columns

`ca2-dataset.csv` holds 1,745 tutor actions; `ca1-dataset.csv` holds the 763 twenty-second field observations (clips) of 89 students, 34 of them off-task (Y) from 22 students. Every action carries the `Unique-id` of its clip, and each clip has one to nine actions (median two). Each of the 15 non-constant `Avg` columns in CA1 equals the clip mean of the matching action column (largest discrepancy 5×10^{-8}), so the clip table is the action log averaged within clips. An average keeps the centre of a clip and discards its count, extremes, spread and composition, which is what the new features recover.

Table 1 lists the checks made before any column was used. Two matter most. `Behaviour` is the label itself, so it is never used. Three clips contain one action logged under a *different* student, all with `time` = −1. CA1 averaged these rows into the clip values; I keep the 14 CA1 inputs exactly as supplied, remove the three actions before engineering new features, and link the students involved, so that all splits use 86 linked student groups.

Table 1
Action-level columns checked before feature engineering

Column(s)	What the check showed	Decision
`Behaviour`	Coded behaviour on every action; `OFF TASK` in all 34 Y clips and no N clip	Never used (copy of the label)
`Coder`	The observer; equals the first part of `Unique-id`	Not a model input; context check only
`Row`	Clips follow `Row` order for only 4 of 89 students	Not used; features are order-free
`lesson`	Two lessons, constant within each student	Excluded (a group label)
`prod`, `cell`	84 skills and 189 widgets, many seen once	Used only through counts (distinct values, repeats)
`pknow-1`, `pchange`	`pknow-1` = −1 exactly when `pchange` = 1, so the two contradict the dictionary	First attempts taken as `pknow-1` ≠ −1
`namea`	3 actions logged under another student, all with `time` = −1	Removed; students linked (86 groups)
`time`	9 actions with `time` = −1 (6 after the removal)	Excluded from timing; counted
9 zero columns	`help`, `choice`, `string`, `number`, `point`, `helppct-up`, `recent8help`, `asymptoteA/B-up`	Not used

1.2 The 24 new features

Each feature group starts from a "just so" story about how off-task behaviour could show up in the log (Table 2): a student who has stopped using the software logs few actions in 20 seconds (A); one long pause hides inside an average but not inside a maximum (B); a student who has mastered a skill may drift off (C); repeated errors suggest struggle rather than disengagement (D); and slow recent actions around a quicker current one may mark a student returning after a pause (E). Group E subtracts each action's own normalized time from the supplied sums over its last three and five actions, the contrast that CA1 built from clip averages (`TimeJump3`, `TimeJump5`); computed per action, it can differ from the CA1 contrast when a clip has more than one action. Each feature is computed with `pandas` from the actions linked to one clip. It uses no label, observer or lesson; the supplied history columns (such as `timelast3SDnormed` or `recent5wrong`) summarize the student's own recent actions, as the CA1 averages already did, and would be available for a new student. The features can therefore be computed before cross-validation without leakage. Because `Row` is not a time order, every feature is order-free. Durations of −1 are excluded from timing summaries and counted in `InvalidTimeCount`; values that remain undefined are filled with training-fold medians.

Table 2
The 24 new features: definitions and single-feature description

Feature	Definition (within one clip)	Median N	Median Y	AUC
A. Activity: an off-task student logs little in 20 s				
`ActionCount`	number of logged actions	2	1	**0.185**
`DistinctCells`	number of different interface widgets (`cell`)	2	1	**0.240**
`DistinctSkills`	number of different skills (`prod`)	2	1	**0.229**
`MaxCellRepeats`	most actions on a single widget	1	1	0.398
B. Timing shape: one long pause can hide inside an average				
`MaxTime`	longest valid action duration (s)	15.0	10.0	0.375
`MinTime`	shortest valid action duration (s)	5.0	8.0	0.598
`TimeSD`	standard deviation of valid durations	2.500	0.000	**0.220**
`TotalTime`	sum of valid durations of the clip's actions (s)[a]	21.0	10.0	**0.297**
`MaxTimeSDnormed`	largest widget-normalized duration	0.181	−0.295	0.335
`MaxTimelast3SDnormed`	largest supplied 3-action sum	0.108	2.119	**0.714**
`MaxTimelast5SDnormed`	largest supplied 5-action sum	−0.173	3.133	**0.739**
`SlowActionCount`	actions with `timeSDnormed` ≥ 1	0	0	0.505
`InvalidTimeCount`	actions with `time` = −1	0	0	0.527
C. Knowledge: students who already know a skill may disengage				
`MeanPknow`	mean BKT estimate `Pknow-2`	0.993	1.000	0.607
`MinPknow`	lowest `Pknow-2`	0.988	1.000	0.636
`FirstAttemptCount`	actions with `pknow-1` ≠ −1	1	1	**0.300**
`FirstAttemptPknow`	mean `pknow-1` over first attempts	0.999	1.000	0.635
D. Errors: struggling or being stuck on one step				
`WrongCount`	number of incorrect actions	0	0	0.452
`BugCount`	number of bug-message actions	0	0	0.462
`MaxRecent5wrong`	largest `recent5wrong`	1	1	0.517
`MaxPrev5Count`	largest `Prev5Count-up`	0	0	0.434
E. Timing contrast: slow recent actions around a quicker current one				
`MaxJump3`	largest per-action `timelast3SDnormed` − `timeSDnormed`[b]	0.350	2.440	**0.724**
`MaxJump5`	largest per-action `timelast5SDnormed` − `timeSDnormed`[b]	0.017	3.580	**0.749**
`MinJump5`	smallest per-action `timelast5SDnormed` − `timeSDnormed`[b]	−1.261	3.580	**0.789**

Note. 729 N and 34 Y clips. AUC of the feature on its own (values below 0.5 mean lower values in Y clips); bold marks |AUC − 0.5| ≥ 0.2. Descriptive only; no feature was dropped on this basis.
[a]A duration can begin before the 20-second window (maximum 288 s), so `TotalTime` is the logged time of the clip's actions, not time inside the window. [b]Computed over actions with a valid duration.

2 Detector Building and Validation

All models are `scikit-learn` pipelines whose first step learns imputation medians from training students only and, where the CA1 inputs are present, adds the three CA1 contrast features. Table 3 lists the feature sets and the candidate models. The soft-voting ensemble averages a linear model, which ranks clips smoothly, and two tree ensembles, which capture thresholds and interactions; whether the ensemble or one of its members is used is decided by the training students. Because only 4.5% of clips are off-task, the decision threshold is also chosen rather than fixed at 0.50.

Table 3
Feature sets and candidate models

Feature set	Content	Features
Old	14 CA1 inputs + 3 CA1 contrasts (`TimeJump3`, `TimeJump5`, `TimePerActRatio`)	17
Old + New	Old + the 24 new features	41
New only	The 24 new features	24
Candidate model	Settings	
Logistic regression	$C = 0.1$; balanced class weights; standardized inputs	
Random forest	300 trees; minimum leaf size 3	
Gradient boosting	100 trees of depth 2; learning rate 0.05; subsample 0.8	
Soft-voting ensemble	Equal-weight average of the three models' probabilities	
Decision threshold	0.05 to 0.90 in steps of 0.05 (18 values)	

Note. The new-feature detector chooses among Old + New and New only (2 sets × 4 models); the old-feature detector uses the Old set (4 models).

Nested cross-validation. The outer loop splits the clips into five folds by the 86 linked student groups (`StratifiedGroupKFold`) and repeats the split with seeds 10 to 14, giving 25 rounds; the code checks in every split that no student or linked group is on both sides (Figure 1). Inside each outer training set, an inner loop of four grouped folds, repeated four times, gives every training clip a held-out probability for every candidate. AUC and the Kappa at each threshold are averaged over the four repeats, and the candidate–threshold pair with the highest (AUC + Kappa) / 2, the criterion the assignment names, is chosen. With only about 27 off-task clips in a training set, the four repeats make this choice steadier than a single inner split. The chosen candidate is trained on the whole outer training set and predicts the outer test students once, so the test students judge the entire procedure, including its choices (Cawley & Talbot, 2010).

Three procedures run on the same 25 splits: the *new-feature detector* (8 candidates), the *old-feature detector* (the same rule on the Old set), and a *CA1 replay*, which applies my CA1 rule (Kappa only, the ten CA1 model configurations, thresholds 0.10–0.60) to the old features. The analysis runs in Python in Google Colab with `scikit-learn` 1.8.0, `NumPy` 2.1.3, `pandas` 2.2.3 and `joblib` 1.6.0, pinned in the notebook.

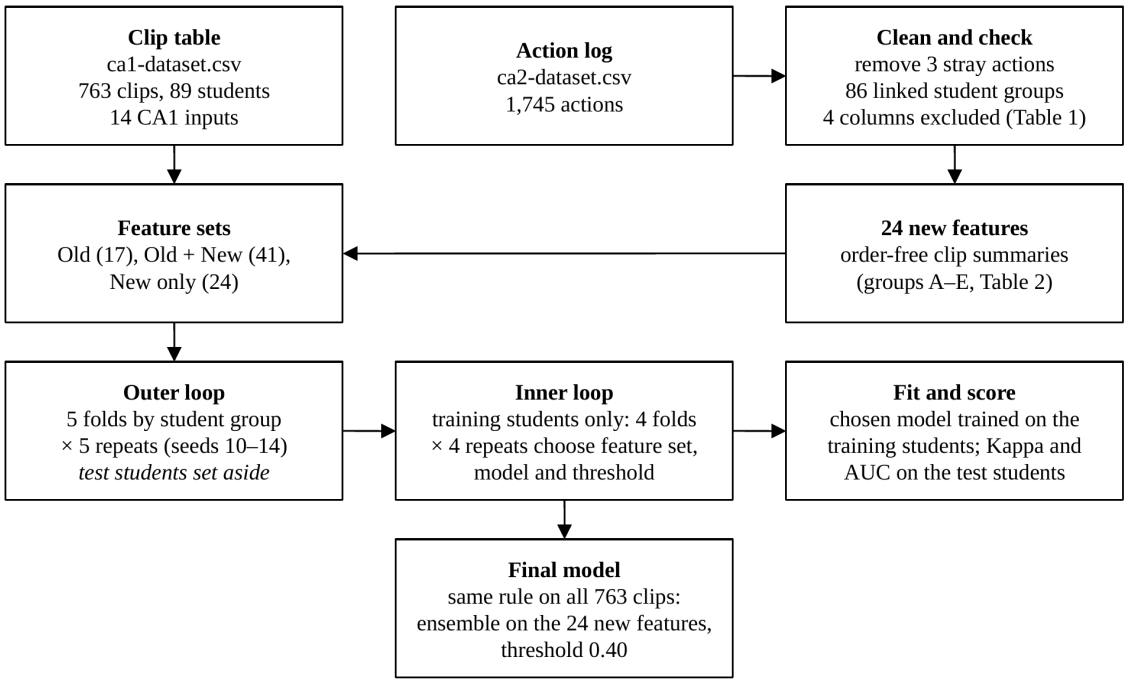


Figure 1
Data preparation and nested, student-grouped cross-validation. Everything learned in a split comes from its training students; the test students only score the result.

3 Results

Pooled Kappa and AUC are computed from all 763 held-out predictions within a repeat and averaged over the five repeats, as in CA1 (Table 4). The new-feature detector is better than the old-feature detector and than the CA1 hand-in on both metrics. With recall 0.465 and precision 0.374, it finds about 16 of the 34 off-task clips; the old-feature detector finds about 11. Its accuracy is below that of always predicting N, because accuracy rewards ignoring a rare class, which is why Kappa and AUC are the measures used here. Inside the 25 training sets, the rule chose the ensemble in 24 rounds and the random forest once, the New only set in 15 rounds and Old + New in 10, and a threshold of 0.35 or 0.40 in 19.

Table 4
Performance on held-out students (means over five repeats)

Detector	Features	Kappa (SD)	AUC (SD)	(AUC + Kappa) / 2	Precision Y	Recall Y	Accuracy
Always predict N	–	0.000	0.500	0.250	–	0.000	0.955
CA1 hand-in[a]	17	0.328 (0.067)	0.777	0.552	0.413	0.318	0.949
CA1 replay[b]	17	0.295 (0.029)	0.786 (0.022)	0.540	0.405	0.271	0.949
Old features	17	0.306 (0.015)	0.750 (0.015)	0.528	0.369	0.318	0.944
New features[c]	24/41	**0.381** (0.067)	**0.884** (0.012)	**0.632**	0.374	0.465	0.940

Note. SD across the five repeats of the same 89 students (descriptive, not a confidence interval). [a]Values reported in CA1, whose folds were grouped by `namea`. [b]The CA1 selection rule re-run on the folds of this table. [c]Feature set chosen inside each training set: New only in 15 rounds, Old + New in 10.

Because the three procedures used the same folds, the gains can be read split by split (Table 5). Against the old-feature detector, the new features raise AUC in every repeat and in 23 of 25 folds, and Kappa in every repeat and in 16 of 25 folds. A test fold holds only about seven off-task clips, so fold-level Kappa is noisy and the repeat-level comparison is the more reliable one. Within each observation context, AUC and Kappa both rise; Kappa stays low in `awagner`, where only 13 of 567 clips are off-task and a few false alarms cost a lot of Kappa.

Table 5
Paired comparison and observation contexts

Panel A. New-feature detector minus each comparator, same 25 folds

	Kappa			AUC		
Comparator	Mean Δ	Repeats higher	Folds higher	Mean Δ	Repeats higher	Folds higher
Old features	+0.075	5/5	16/25	+0.133	5/5	23/25
CA1 replay	+0.086	5/5	15/25	+0.098	5/5	23/25

Panel B. Within each observation context

	`awagner` (567 clips, 13 Y)		`jkalka` (196 clips, 21 Y)	
Detector	AUC	Kappa	AUC	Kappa
CA1 replay	0.784	0.244	0.761	0.327
Old features	0.724	0.168	0.757	0.410
New features	0.898	0.255	0.856	0.495

Note. "Higher" counts repeats (of 5) and folds (of 25) in which the new-feature detector scored higher; the counts describe this validation, not a significance test. The observer is never a model input.

4 Final Model and Interpretation

The final detector applies the same selection rule to all 763 clips. It chose the soft-voting ensemble on the 24 new features with a threshold of 0.40 (inner Kappa 0.432); the ensemble on Old + New at 0.35 was a close second (inner score 0.653 against 0.658), matching the split choice between the two sets during cross-validation. Its estimated performance for new students is the cross-validated result in Table 4, because the cross-validation evaluated exactly this way of building the model. The model leans most on timing shape and the timing contrasts (Table 6). Together with Section 1.2, this suggests that an off-task clip usually holds a single logged action whose recent window is slow; one reading, which the log alone cannot confirm, is a student returning after a pause. CA1 could see this only through averages; the action count and the per-clip extremes make it explicit, consistent with Sao Pedro et al. (2012), who found that features designed around the construct can yield better models with less information.

Table 6
Feature importance in the final model

Panel A. Ten most important features

Feature	Group	Importance
MaxTimeSDnormed	B	0.129
TotalTime	B	0.104
MinJump5	E	0.099
MaxJump3	E	0.067
ActionCount	A	0.064
MaxJump5	E	0.059
MaxTimelast5SDnormed	B	0.047
MaxTimelast3SDnormed	B	0.044
FirstAttemptPknow	C	0.043
DistinctSkills	A	0.040

Panel B. Summed by group

Group	Sum
B. Timing shape (9)	0.440
E. Timing contrast (3)	0.225
A. Activity (4)	0.158
C. Knowledge (4)	0.104
D. Errors (4)	0.074

Note. Mean of the three ensemble members' normalized importances (impurity importance for the tree models, absolute standardized coefficients for logistic regression). Group sums are a heuristic summary of what the model leans on, not shares of predictive power; correlated features share importance.

Scope of the results. At the selected threshold, about 37% of the clips the detector flags are off-task, more than eight times the 4.5% base rate, and it finds about half of all off-task clips. For a particular classroom use, the threshold could be moved to reflect the relative cost of a missed off-task clip and an unnecessary prompt. The estimates rest on 34 off-task clips from 22 students, and Kappa varies across the five repeats (SD 0.067). Because the features and candidate models were designed with this dataset, data from new students would provide the strongest confirmation of the result. Applying the detector to new logs requires timing columns normalized in the same way, and its first-attempt features follow the documented meaning of `pknow-1`.

5 Submitted Files

Table 7
Submitted files

File	Content
Mengchen_Li_6191_CA2.pdf	This report
Mengchen_Li_6191_CA2.ipynb	Complete code with comments and outputs (also shared as a Colab link: **https://colab.research.google.com/drive/1Jk-GgY9xMDOtUPagfiVvdAo8idjYxaYR?usp=sharing**)
Mengchen_Li_6191_CA2_old_features_data.csv	Old feature set input: identifiers, label, 14 CA1 inputs (763 × 17)
Mengchen_Li_6191_CA2_new_features_data.csv	As above plus the 24 new features (763 × 41)
Mengchen_Li_6191_CA2_final_model.joblib	Final model, imputation values, features, threshold (0.40), versions, cross-validated performance
Mengchen_Li_6191_CA2_heldout_predictions.csv	Supporting: every held-out prediction behind Tables 4 and 5

6 AI Use Statement

I used generative AI tools (Claude and ChatGPT) as assistants in this assignment. They helped me write and debug the Python code, review the methodology, check and interpret the outputs, and draft and revise parts of this report. I reviewed the code and every reported result, which comes from a complete run of the submitted notebook, and I take responsibility for the analysis and the final text.

References

Cawley, G. C., & Talbot, N. L. C. (2010). On over-fitting in model selection and subsequent selection bias in performance evaluation. *Journal of Machine Learning Research*, *11*, 2079–2107. https://www.jmlr.org/papers/v11/cawley10a.html

Sao Pedro, M. A., Baker, R. S. J. d., & Gobert, J. D. (2012). Improving construct validity yields better models of systematic inquiry, even with less information. In J. Masthoff, B. Mobasher, M. C. Desmarais, & R. Nkambou (Eds.), *User modeling, adaptation, and personalization: UMAP 2012* (Lecture Notes in Computer Science, Vol. 7379, pp. 249–260). Springer. https://doi.org/10.1007/978-3-642-31454-4_21